Availability Collection

Tags	snmp
Team Owner	The Collectables
Status	ACTIVE
Last Modified	May 7, 2024

- Introduction
- Diagnosis and Mitigation
  - finding ICDA apps
  - analyzing sigterms
- Theory
  - populate device query
- Post Mortem Articles
  - 2024-11-21 EM-71286: device availability reporting blank
  - 2024-08-15 EM-66443: Navy MUOS - Availability Collect w/ 1min frequency has odd variance in collection times
  - 2024-05-17 EM-65040: Ping device monitor failed to alert
  - 2024-05-07 EM-64863: Several devices failing SNMP availability checks intermittently
  - 2023-07-17 EM-59710:
  - 2020-04-17 Insight Digital Realty Availability Sigterms
- Silo Log Messages
  - Log Pattern Analysis
    - main process
      - stanza:a complete run
    - child process
      - stanza:collecting against one IP with success
      - stanza:collecting against one IP address with failure
- Known Issues
  - The one straggler problem

Introduction

Diagnosis and Mitigation

finding ICDA apps

Sometimes availability collection will fail because of an app configured to run as an ICDA app with v_dynamic_app.data_model value of 1. ICDA possible data_models are taken from this list:

```
# this list lives in G3code/backend/silo_common/global_definintions.py
ICDA_DATA_MODEL_AVAIL = 1
##############################################################################
# ICDA Data Models
##############################################################################
# FIXME_INFO_TYPE: icda_data_models {
ICDA_DATA_MODEL_AVAIL = 1
ICDA_DATA_MODEL_CBQOS_PERF = 2
ICDA_DATA_MODEL_DNS = 3
ICDA_DATA_MODEL_FS_PERF = 4
ICDA_DATA_MODEL_PROC_PERF = 5
ICDA_DATA_MODEL_SERV_PERF = 6
ICDA_DATA_MODEL_PORT = 7
ICDA_DATA_MODEL_SNMP_DETAIL = 8
ICDA_DATA_MODEL_IF_PERF = 9
ICDA_DATA_MODEL_CV = 10
ICDA_DATA_MODEL_TV = 11
ICDA_DATA_MODEL_FS_INV = 12
ICDA_DATA_MODEL_CBQOS_INV = 13
ICDA_DATA_MODEL_IF_INV = 14
ICDA_DATA_MODEL_PROC_INV = 20
ICDA_DATA_MODEL_SERV_INV = 21

```

When diagnosing a customer system's dynamic apps, the query in this section can dump all app alignments: [Dump (did,app_id) alignments: just grab everything]

The rows in that dump file where the "data_model" column is 1 will be ICDA dynamic apps run as part of availability collection.

analyzing sigterms

When availability collection processes are sigterming on a customer system, you can use a query like to find out how often and on which CUs are occurring:

```
silo_mysql -e "select * from master_logs.system_messages where message like '%SIGTERM%' or message like '%availability%' and date_edit>=now() - interval 10 day" | gzip -c > syslogs.txt.gz

```

This would let you see, for the past 10 days, the availability sigterm messages like this:

```
4719875 3       2020-04-16 03:35:15     43      1: Data Collection: Availability process did not complete in its allowed timeslot, SIGTERM'ing. Run time: 32m (max 30m) 1       0000-00-00
```

```
00:00:00     0

```

as well as these messages (this example is taken from silo.log, and would look different when captured from the syslogs):

```
"2020-04-16 14:34:02,464","INFO","4628","10","shared","248","Data Collection: Availability task list at termination: [4294]"

```

The task list at termination is the list of DIDs that were not collected by the time the availability process was terminated. In cases where one or a small number of DIDs are holding up the rest, it would be easy to determine that set by looking at these messages from the syslog.

These messages are not always paired. That is, you will sometimes see more "sigterm" messages than "task list at termination" messages. That's because you will only see "task list at termination" messages for processes that didn't finish collecting against all devices in their list. It is possible for a collection process to finish collection but get stuck in a later stage (like post processing).

Theory

populate device query

The query text lives in this section of code (G3code/backend/silo_collect/collect_avail.py):

```
class avail_collection_dal(object):
    @staticmethod
    def get_device_records(dbc, logger, mid, sql_device_filter):
        dbc.execute_d(
            """
            SELECT dev.did, dev.snmp_cred_id, dev.ip, dev.hostname, dev.active, dev.avail_proto, dev.avail_port,
              dev.latency_proto, dev.latency_port, dev.alert_avail_and_latency_fail,
              '' AS device_up, '' AS latency
            FROM collector_state.V_device AS dev
            JOIN master.definitions_dev_classes AS dev_type ON (dev_type.class_type = dev.class_type)
            WHERE %s AND (dev.active & %%s) = 0
              AND dev.avail_proto IN %%s
              AND dev.pid = %%s
              AND dev_type.virtual = 0
              AND dev.dhcp = 0
            """ % (sql_device_filter,),
            (SILO_ACTIVE_MASK | SILO_MAINT_COLLECTION_MASK, (SILO_TCP, SILO_UDP, SILO_ICMP, SILO_BULK_ICMP), mid),
            log = True, logger = logger
        ) #only user-disabled devices are filtered out.

        return dbc.fetchall()

```

If we replace symbolic constants with literals, and rearrange to show the results for all pids, we have:

```
            SELECT dev.pid, dev.did, dev.snmp_cred_id, dev.ip, dev.hostname, dev.active, dev.avail_proto, dev.avail_port,
              dev.latency_proto, dev.latency_port, dev.alert_avail_and_latency_fail,
            FROM collector_state.V_device AS dev
            JOIN master.definitions_dev_classes AS dev_type ON (dev_type.class_type = dev.class_type)
            WHERE
              dev.active & 9 = 0
              AND dev.avail_proto IN (0,1,2,3)
              AND dev_type.virtual = 0;

```

Which we can collapse into this one liner, to dump all availability collection from the CDB's point of view

```
silo_mysql -e "SELECT dev.pid, dev.did, dev.snmp_cred_id, dev.ip, dev.hostname, dev.active, dev.avail_proto, dev.avail_port, dev.latency_proto, dev.latency_port, dev.alert_avail_and_latency_fail FROM collector_state.V_device AS dev JOIN master.definitions_dev_classes AS dev_type ON (dev_type.class_type = dev.class_type) WHERE dev.active & 9 = 0 AND dev.avail_proto IN (0,1,2,3) AND dev_type.virtual = 0" > cdb_avail_pop_dev.txt
```

We can also run this query against all CUs (From the CDB) through this command line to see the system wide view from CUs:

```
./run_cu_query.py run "SELECT dev.pid, dev.did, dev.snmp_cred_id, dev.ip, dev.hostname, dev.active, dev.avail_proto, dev.avail_port, dev.latency_proto, dev.latency_port, dev.alert_avail_and_latency_fail FROM collector_state.V_device AS dev JOIN master.definitions_dev_classes AS dev_type ON (dev_type.class_type = dev.class_type) WHERE dev.active & 9 = 0 AND dev.avail_proto IN (0,1,2,3) AND dev_type.virtual = 0" > cu_avail_pop_dev.txt
```

comparing these two results, and finding discrepancies, could indicate that config push is failing to update some CUs.

The run_cu_query.py script can be found here:

```
https://code.eng.sciencelogic.com/projects/TOOL/repos/em7_diag/browse/run_cu_query.py

```

Post Mortem Articles

2024-11-21 EM-71286: device availability reporting blank

ts_guide_to_jira_link

Link: EM-71286: device availability reporting blank CLOSED

From the description:

```
Observed Behavior: Availability UHE IPAddressError: Address passed was not valid UHE

Expected Behavior: No Availability UHE IPAddressError: Address passed was not valid UHE

Summary of troubleshooting steps taken so far and the current status:

Appears to be the same issue as https://sciencelogic.atlassian.net/browse/EM-59710#icft=EM-59710, if eng can please check the UHE and confirm?
We tried to follow KB-12391 which invloves making changes to '/opt/em7/lib/python/sl_availability/collect_avail.py' in 11.3, however 12.1.1 the file is compiled .pyc so we cannot edit it. Looking for a potential
 file if this is indeed the same as https://sciencelogic.atlassian.net/browse/EM-59710#icft=EM-59710, or confirmation the customer must update SL1 to resolve this. Or another method of identifying the devices causing
 this issue and providing a workaround for those devices?

Currently they have three collectors which are facing this issue.

I’ve asked customer to Enable internal Process Data Collection: Availability in debug mode and provide us /var/log/sl1/availability_collect.log. Currently waiting for their response

Reason for escalation:
Datacom reached about this issue earlier and an escalation was already raised:
https://sciencelogic.atlassian.net/browse/EM-66894

But it didn’t proceed further as customer didn’t provide the availability logs.

Reason for escalation being critical:

Customer has raised a S2 case mentioning that:
Above error result in the device availability report to be empty which in turn is impacting their SLA to accurately report to their customer

the error also appears to get resolved when they update the collectors from 12.1.1 to 12.1.2. However they still have about ~10 days before they complete all the collector updated to 12.1.2
```

2024-08-15 EM-66443: Navy MUOS - Availability Collect w/ 1min frequency has odd variance in collection times

ts_guide_to_jira_link

Link: EM-66443: Navy MUOS - Availability Collect w/ 1min frequency has odd variance in collection times CLOSED

From the description:

```
Observed Behavior:

Variance of 3-4 seconds, observed going as high as 100 seconds, in the availability collections.

Expected Behavior:

Should be closer to exactly 1 minute each time

```

```
Summary of troubleshooting steps taken so far and the current status:
Mainly looking to see if there is anything we can do to get the process to be closer to 1 minute in each cycle.
Slack thread with collectables:
https://sciencelogic.slack.com/archives/C04L444TLG2/p1722631317590659
Note, this customer has a 1 minute availability requirement. The process is set to poll every 1 minute, they cannot and will not change it back to 5 minutes.
Performed some of the basic 'sigterm' triage steps (batch factor to 2, etc.)
Collector is not overloaded, and this stack is very small. There are no IC apps aligned to any devices on the entire stack. We also have about 30 hours of availability debug logs.

Customer had this to say when asked about why this is setup like this:
If we are truly certain that this would need to go back to 5 minutes then we would need a very detailed justification and reason as to why this would change from previous discussions/agreements.
As for Device discovery, we discover all the devices via IP address. We do this as a targeted single discovery per device.
Not sure what SNAP availability is but we do rely heavily on the PING and SNMP to tell us if the device is alive and available from SL1's perspective.
From the debug logs, does it show a large amount of devices not responding? I guess what would be considered a large amount as we have 2 Data Collectors configured in a Collector Group in our system with ~135 devices managed. This seems like very light load given the System Requirements indicated on the SL1 website.
Lastly, for the credential timeout/retries, we use the default values for these. We have not modified these values from any of the defaults currently.
```

slack thread: https://sciencelogic.slack.com/archives/C04L444TLG2/p1722631317590659 Connect your Slack account

2024-05-17 EM-65040: Ping device monitor failed to alert

ts_guide_to_jira_link

Link: EM-65040: Ping device monitor failed to alert CLOSED

From the description:

```
Observed Behavior: System Availability is not being shown properly in the device Performance graph. It looks like it is being collected.
Expected Behavior: should be able to see some metrics for System Availability for these devices on this CUG, It is collecting SNMP data and other DA data.
```

2024-05-07 EM-64863: Several devices failing SNMP availability checks intermittently

ts_guide_to_jira_link

Link: EM-64863: Several devices failing SNMP availability checks intermittently CLOSED

From the description:

```
Observed Behavior: Some devices on multiple collectors in different collector groups are failing availability checks with SNMP. Customer says that the devices were 100% available during the times that they are showing unavailable.
Expected Behavior: Devices that are available do not fail SNMP availability checks.
Summary of troubleshooting steps taken so far and the current status:
```

```
Concurrent SNMP collection is the default collections for the collector groups. Removed concurrent collections from a single collector group and it did appear that there were less availability issues but they were
still present. When the devices become unavailable if they are rediscovered by clicking on the binoculars the devices become available again. Gathered the data collection availability debug logs from the collectors
that appear to be impacted. No SIGTERMS for data collection observed on the impacted collectors.
```

2023-07-17 EM-59710: 💔EM-59710: Most Availability collections failing with unhandled exceptions caused by only a few IPs CLOSED

ts_guide_to_jira_link

Link: 💔EM-59710: Most Availability collections failing with unhandled exceptions caused by only a few IPs CLOSED

From the description:

```
Observed Behavior: Entire availability collection fails when it encounters a bad IP address

Expected Behavior: Only the device with the bad IP should fail

Summary of troubleshooting steps taken so far and the current status:

Availability collections for MOST devices were failing with unhandled exceptions for only a few IPs, which are stopping the entire availability collection process:
/opt/em7/lib/python/sl_availability/collect_avail.py", line 593, in _run_collect_async self.devices[device] = result.get()
File "/usr/lib64/python2.7/multiprocessing/pool.py", line 554, in get raise self._value ChildProcessException: Traceback (most recent call last):
 File "/opt/em7/lib/python/sl_process/exceptions.py", line 43, in new_f res = func(*args, **kwargs)
 File "/opt/em7/lib/python/sl_availability/collect_avail.py", line 165, in collect ip_object = ip(details["ip"])
 File "/opt/em7/lib/python/sl_network/network.py", line 120, in __init__ raise IPAddressError("Address passed was not valid") IPAddressError: Address passed was not valid

From the exceptions:
File "/opt/em7/lib/python/sl_availability/collect_avail.py", line 165, in collect#012 ip_object = ip(details["ip"])#012

Looking at the related code on the collector in /opt/em7/lib/python/sl_availability/collect_avail.py:
    else:  # collect on a single ip address.
        try:
            ip_object = ip(details["ip"])
        except IPAddressError:
            logger.warning(
                "IP address of '%s' for did %s is invalid.  Not collecting availability for device",
                details["ip"],
                details["did"],
            )
            return details

```

```
So based on the code above I would expect that it should be throwing warnings as noted in above code instead of being unhandled and breaking the collection...

It is looking for an IPAddressError:
except IPAddressError:

and the network code (network.py", line 120) is raising the same exception type
raise IPAddressError("Address passed was not valid")

Which means it should be caught by the except and be handled gracefully with the warning message, but for some reason it is not...

As a workaround for now, since the "collect_avail.py" file is editable on the collector, we tested changing it from the IP Address specific except to a generic except:
#except IPAddressError:
except:

That successfully worked around the problem and availability collections started working as expected.
We are now actually getting the "Not collecting availability for device" warnings that I expected to see when it hits problem devices.

Looking deeper into this, I reviewed the code for this file that bitbucket has tagged for 11.3.0 and it seems that this should really look like this and be on different code lines than the exceptions show:
    else:  #collect on a single ip address.
        try:
            ip_object = ip(details['ip'])
        except scexcept.IPAddressError as e:
            logger.warning("IP address of '%s' for did %s is invalid.  Not collecting availability for device" % (details['ip'], details['did']))
            return details

Looking at a few our 11.3 lab machines (I didn't check this on customer-side but saw it on every lab machine i checked), I am seeing that there are 2 files... one with py and one with pyc...
$ ls -la /opt/em7/lib/python/sl_availability/collect_avail.py*
-rw-r----- 1 root s-em7-srv 35062 Apr 13 22:58 /opt/em7/lib/python/sl_availability/collect_avail.py
-rw-r----- 1 root s-em7-srv 25913 Apr 13 22:58 /opt/em7/lib/python/sl_availability/collect_avail.pyc

Is it possible that an older .py file is somehow overriding an updated pyc file because of this?
```

2020-04-17 Insight Digital Realty Availability Sigterms

ts_guide_to_jira_link

Link: [Insight Digital Realty]

This [section] of that article walks through an analysis using the silo.log debug messages from the availability collection main process and worker processes. And this [section] shows how an ICDA availability app was the root cause of the escalation.

Silo Log Messages

Log Pattern Analysis

main process

stanza:a complete run

```
"2021-04-01 20:02:03,059","DEBUG","21510","10","proc_pda","256","Got proc virtual memory limit from silo.conf:None"
"2021-04-01 20:02:03,059","DEBUG","21510","10","proc_pda","268","No or Incorrect virtual memory limit from silo.conf:proc_id=10 : None"
"2021-04-01 20:02:03,071","DEBUG","21510","10","proc_pda","190","Logging started for Data Collection: Availability"
"2021-04-01 20:02:03,071","DEBUG","21510","10","proc_pda","215","importing pda tracker"
"2021-04-01 20:02:03,073","DEBUG","21510","10","proc_pda","500","Started tracking process: 501 10 20538cd2932511eb9b0c005056a555c7 "
"2021-04-01 20:02:03,073","DEBUG","21510","10","proc_pda","374","Trying to get system settings"
"2021-04-01 20:02:03,074","DEBUG","21510","10","shared","184","Official collection time: 2021-04-01 20:02:00 timestamp:1617307320"
"2021-04-01 20:02:03,095","DEBUG","21510","10","database","173","QL: query:      SELECT did,
"2021-04-01 20:02:03,096","DEBUG","21510","10","database","174","QL: dbc id: 139898988522320"
"2021-04-01 20:02:03,096","DEBUG","21510","10","database","175","QL: query time: 0.0212509632111"
"2021-04-01 20:02:03,096","DEBUG","21510","10","database","176","QL: lastrowid:  0"
"2021-04-01 20:02:03,096","DEBUG","21510","10","database","177","QL: rowcount:   2447"

"2021-04-01 20:02:03,163","DEBUG","21510","10","collect_avail","468","{0: {'ping_packet_size': 56.0, 'avail_ping_req_pct': 80.0, 'avail_ping_count': 1.0, 'latency_thresh': 100.0}, 32840: {'ping_packet_size': None, 'avail_ping_req_pct': 50.0, 'avail_ping_count': 2.0, 'latency_thresh': 200.0}, ... 32461: {'ping_packet_size': None, 'avail_ping_req_pct': 50.0, 'avail_ping_count': 2.0, 'latency_thresh': 200.0}}

"2021-04-01 20:02:03,179","DEBUG","21510","10","database","173","QL: query:      SELECT dev.did, dev.snmp_cred_id, dev.ip, dev.hostname, dev.active, dev.avail_proto, dev.avail_port,
"2021-04-01 20:02:03,180","DEBUG","21510","10","database","174","QL: dbc id: 139898988522320"
"2021-04-01 20:02:03,180","DEBUG","21510","10","database","175","QL: query time: 0.00521087646484"
"2021-04-01 20:02:03,180","DEBUG","21510","10","database","176","QL: lastrowid:  0"
"2021-04-01 20:02:03,180","DEBUG","21510","10","database","177","QL: rowcount:   278"
"2021-04-01 20:02:03,180","DEBUG","21510","10","collect_avail","503","Found some overrides did:25496"
"2021-04-01 20:02:03,180","DEBUG","21510","10","collect_avail","511","ping_packet_size is None, using system default"
"2021-04-01 20:02:03,180","DEBUG","21510","10","collect_avail","514","avail_ping_req_pct is 50.0"
"2021-04-01 20:02:03,180","DEBUG","21510","10","collect_avail","514","avail_ping_count is 2.0"
"2021-04-01 20:02:03,180","DEBUG","21510","10","collect_avail","514","latency_thresh is 200.0"
"2021-04-01 20:02:03,180","DEBUG","21510","10","collect_avail","525","Final Thresholds for did 25496: latency 200.0, num pings 2.0, avail ping percent 50.0, ping size 56.0"
...
```

```
"2021-04-01 20:02:03,330","DEBUG","21510","10","collect_avail","528","avail sys row count: 278"
"2021-04-01 20:02:03,337","DEBUG","21510","10","database","173","QL: query:      SELECT
"2021-04-01 20:02:03,337","DEBUG","21510","10","database","174","QL: dbc id: 139898988522320"
"2021-04-01 20:02:03,337","DEBUG","21510","10","database","175","QL: query time: 0.00665903091431"
"2021-04-01 20:02:03,337","DEBUG","21510","10","database","176","QL: lastrowid:  0"
"2021-04-01 20:02:03,337","DEBUG","21510","10","database","177","QL: rowcount:   0"
"2021-04-01 20:02:03,337","DEBUG","21510","10","shared","200","collection_process.get_num_spawn: cpu_count = 4"
"2021-04-01 20:02:03,337","DEBUG","21510","10","shared","212","collection_process.get_num_spawn: num_devices = 278, min_spawn = 2, max_spawn = 8, throttle = 9"
"2021-04-01 20:02:03,370","DEBUG","21510","10","shared","191","collection_process.start_jobservers: num_procs = 8, worker_task_limit = None, number of requests = 278"
"2021-04-01 20:02:03,388","DEBUG","21510","10","shared","193","Started 8 worker processes, task limit = None"
"2021-04-01 20:02:03,388","UIDEBUG","21510","10","collect_avail","391","*** First collection pass"
"2021-04-01 20:02:03,388","DEBUG","21510","10","collect_avail","412","Running collect asynchronously"
"2021-04-01 20:02:03,388","UIDEBUG","21510","10","collect_avail","415","appending job to queue for device 28263"
"2021-04-01 20:02:03,389","UIDEBUG","21510","10","collect_avail","415","appending job to queue for device 25998"
...
"2021-04-01 20:02:45,535","UIDEBUG","21510","10","collect_avail","435","got results for device 27618"
"2021-04-01 20:02:45,535","UIDEBUG","21510","10","collect_avail","435","got results for device 28052"
"2021-04-01 20:02:45,535","DEBUG","21510","10","collect_avail","440","Number of devices left: 0"
"2021-04-01 20:02:46,536","UIDEBUG","21510","10","collect_avail","395","*** Second collection pass (retrying failed availability attempts)"
"2021-04-01 20:02:46,536","DEBUG","21510","10","collect_avail","412","Running collect asynchronously"
"2021-04-01 20:02:46,536","UIDEBUG","21510","10","collect_avail","415","appending job to queue for device 28263"
"2021-04-01 20:02:46,536","UIDEBUG","21510","10","collect_avail","415","appending job to queue for device 25998"
...
"2021-04-01 20:02:46,610","UIDEBUG","21510","10","collect_avail","435","got results for device 25998"
"2021-04-01 20:02:46,610","UIDEBUG","21510","10","collect_avail","435","got results for device 28264"
...
"2021-04-01 20:02:51,635","UIDEBUG","21510","10","collect_avail","435","got results for device 28492"
"2021-04-01 20:02:51,635","UIDEBUG","21510","10","collect_avail","435","got results for device 26558"
"2021-04-01 20:02:51,636","DEBUG","21510","10","collect_avail","440","Number of devices left: 0"
"2021-04-01 20:02:52,637","DEBUG","21510","10","collect_avail","398","Joining the process pool"
```

```
"2021-04-01 20:02:52,707","DEBUG","21510","10","collect_avail","263","checking latency for device 28263.  collected 46.8975, threshold 200.0: alert=False"
"2021-04-01 20:02:52,708","DEBUG","21510","10","collect_avail","263","checking latency for device 25998.  collected 4.753, threshold 200.0: alert=False"
...
"2021-04-01 20:02:52,725","DEBUG","21510","10","collect_avail","263","checking latency for device 27618.  collected 55.262, threshold 200.0: alert=False"
"2021-04-01 20:02:52,726","DEBUG","21510","10","outage_objects","87","Checking 28263 for outage of type 0"
"2021-04-01 20:02:52,727","DEBUG","21510","10","outage_objects","94","no outage for 28263 of type 0"
"2021-04-01 20:02:52,727","DEBUG","21510","10","outage_objects","87","Checking 28263 for outage of type 2"
"2021-04-01 20:02:52,727","DEBUG","21510","10","outage_objects","94","no outage for 28263 of type 2"
...
"2021-04-01 20:02:52,793","DEBUG","21510","10","emitters","166","Device 25991 Alerting: msg_id = 267, msg_val= ICMP Ping, ytype= 0, yid=0, yname="
"2021-04-01 20:02:52,794","DEBUG","21510","10","emitters","166","Device 25991 Alerting: msg_id = 183, msg_val= No Response, ytype= 0, yid=0, yname="
...
"2021-04-01 20:02:52,804","DEBUG","21510","10","database","173","QL: query:      SELECT
"2021-04-01 20:02:52,804","DEBUG","21510","10","database","174","QL: dbc id: 139898988522320"
"2021-04-01 20:02:52,804","DEBUG","21510","10","database","175","QL: query time: 0.00565195083618"
"2021-04-01 20:02:52,804","DEBUG","21510","10","database","176","QL: lastrowid:  79855528"
"2021-04-01 20:02:52,804","DEBUG","21510","10","database","177","QL: rowcount:   0"
"2021-04-01 20:02:52,804","DEBUG","21510","10","collect_avail","571","icda applicable devices found: ()"
"2021-04-01 20:02:52,804","DEBUG","21510","10","collect_avail","304","do_storage called"
"2021-04-01 20:02:53,036","DEBUG","21510","10","collect_avail","323","do_storage finished"
"2021-04-01 20:02:53,037","DEBUG","21510","10","proc_pda","528","tracking update: 501 10 20538cd2932511eb9b0c005056a555c7 "
"2021-04-01 20:02:53,039","DEBUG","21510","10","proc_pda","545","Stopped tracking process: 501 10 20538cd2932511eb9b0c005056a555c7 "
"2021-04-01 20:02:53,039","DEBUG","21510","10","proc_pda","307","Logging stopped for Data Collection: Availability"

```

child process

stanza:collecting against one IP with success

```
"2021-04-01 20:02:03,448","DEBUG","21654","10.PoolWorker-5","logger","194","Returning NEW logger instance."
"2021-04-01 20:02:03,448","DEBUG","21654","10.PoolWorker-5","collect_avail","93","Availability Collect for ip(s) 10.75.13.253"
"2021-04-01 20:02:03,448","UIDEBUG","21654","10.PoolWorker-5","collect_avail","135","Using ICMP for latency and availability measurement"
"2021-04-01 20:02:03,448","UIDEBUG","21654","10.PoolWorker-5","tools","108","ping command: /bin/ping -c2.0 -W 1 10.75.13.253 "
"2021-04-01 20:02:04,581","UIDEBUG","21654","10.PoolWorker-5","tools","110","ping response: PING 10.75.13.253 (10.75.13.253) 56(84) bytes of data.
"2021-04-01 20:02:04,581","DEBUG","21654","10.PoolWorker-5","tools","134","packet received is 100 Threshold: 50.0"
"2021-04-01 20:02:04,581","DEBUG","21654","10.PoolWorker-5","tools","136","Device Available"
```

```
"2021-04-01 20:02:04,582","UIDEBUG","21654","10.PoolWorker-5","collect_avail","169","Latency = 53.7815"
"2021-04-01 20:02:04,583","DEBUG","21654","10.PoolWorker-5.Extended_Internal_Collection","logger","194","Returning NEW logger instance."
"2021-04-01 20:02:04,583","UIDEBUG","21654","10.PoolWorker-5","collect_avail","192","Device up:  True"
```

stanza:collecting against one IP address with failure

```
"2021-04-01 20:02:26,367","DEBUG","21654","10.PoolWorker-5","logger","194","Returning EXISTING logger instance."
"2021-04-01 20:02:26,367","DEBUG","21654","10.PoolWorker-5","collect_avail","93","Availability Collect for ip(s) 10.92.22.100"
"2021-04-01 20:02:26,367","UIDEBUG","21654","10.PoolWorker-5","collect_avail","162","Using SNMP Request for latency measurement"
"2021-04-01 20:02:26,367","UIDEBUG","21654","10.PoolWorker-5","collect_avail","165","Successfully obtained SNMP connection to device"
"2021-04-01 20:02:26,367","UIDEBUG","21654","10.PoolWorker-5","tools","274","Calling getnext on .1.3.6.1"
"2021-04-01 20:02:29,371","UIDEBUG","21654","10.PoolWorker-5","tools","286","Failed.  SNMP Error: Timeout, index [-24], error code [0], resp [[('.1.3.6.1.', None)]]"
"2021-04-01 20:02:29,371","UIDEBUG","21654","10.PoolWorker-5","collect_avail","169","Latency = 0"
"2021-04-01 20:02:29,371","UIDEBUG","21654","10.PoolWorker-5","collect_avail","172","availability check with 2.0 ping(s), required 50.0% replies, and a size of 56.0"
"2021-04-01 20:02:29,371","UIDEBUG","21654","10.PoolWorker-5","tools","108","ping command: /bin/ping -c2.0 -W 1 10.92.22.100 "
"2021-04-01 20:02:31,379","UIDEBUG","21654","10.PoolWorker-5","tools","110","ping response: PING 10.92.22.100 (10.92.22.100) 56(84) bytes of data.
"2021-04-01 20:02:31,380","DEBUG","21654","10.PoolWorker-5","tools","142","Device OBVIOUSLY NOT Available"

```

Known Issues

The one straggler problem

If all devices are collected against but one, that is able to evade timeouts and run out the clock so that the availability poll is sigtermed, then none of the data collected by the main process from the workers will be packaged into storage objects. This will manifest as availability data gaps in all the devices assigned to that collector.